

CAD Programming Assignment 4

Open-source Analog Sizing Tool: AutoCkt

December 2025

1. Objective

The assignment is divided into two stages.

- **Stage 1:** Follow the tutorial to install the open-source tool AutoCkt, set up its environment, and successfully run all the functionalities described in the manual.
- **Stage 2:** For a given analog circuit, apply the agent trained by AutoCkt to automatically tune transistor parameters to meet the target specifications, and output the final SPICE netlist of the tuned circuit.

In this stage, you will design a Python program to implement this functionality; see Section 2, Program Interface, for details.

2. Program Interface

2.1. Execution

```
$ conda activate autockt
$ python sizing_[student_ID].py --model <model> --spec <spec_file>
```

2.2. Input: two primary input files

- Agent trained by AutoCkt (<model>)
Refer to "/abs/path/to/checkpoint" in the step 7 of section "AutoCkt" in the manual.
- Target specifications of the circuit (<spec_file>)
A JSON file contains the target specifications. It will include only the following four items: gain_min (minimum gain), ibias_max (maximum Ibias), phm_min (minimum phase margin), ugbw_min (minimum unit-gain bandwidth)

2.3. Output: Upon a successful execution, one primary output file

- SPICE netlist of the final tuned circuit (final_design.cir)
- See Section 4, Important Notice, for details.

3. Submission

Please upload a compressed file **[student_ID]_HW4.zip** includes following:

3.1. Stage 1: Screenshot of "Verifying AutoCkt"

Refer to the step 7 of section “AutoCkt” in the manual.

Capture a screenshot and **include it in your report**. The screenshot must at least show the end of the program execution and the lines for "episode reward", "specs reached", and "num specs reached" (see the figure below for reference).

3.2. Stage 2 : Programming files (new version)

- sizing_[student_ID].py
- Trained agent: model_[student_ID]

```
-----
params = [17 14 87 13 5 25 68]
specs: [3.83609010e+02 2.46211100e-04 6.05622524e+01 1.13321670e+07]
ideal specs: [3.99000000e+02 1.04731608e-03 6.00000000e+01 6.83641821e+06]
re: 10
-----
[array([1]), array([0]), array([2]), array([0]), array([0]), array([0]), array([2])]
10
True
Episode reward 0.8766924501795419
Specs reached: 44/6
Num specs reached: 44/50
In [2]:
```

The agent you trained using AutoCkt, i.e. checkpoint.

3.3. Report

- Screenshot of "Verifying AutoCkt"
- What did you learn from this assignment?
- Problems and solutions.

4. Important Notice

- [student_ID] must be replaced with your actual student ID.
- Stage 1: set **num_val_specs** to **100**, and the **success rate must reach at least 85%**, i.e., the number of specs reached must be at least 85.

- Stage 2: Python Package

- In demo, TAs will only use the default python package installed in AutoCkt (environment.yml), so **you don't need to install other python packages.**
- Stage 2: Boundary conditions
 - gain_min: 200~400
 - ibias_max: 0.0001~0.01
 - phm_min: 60
 - ugbw_min: 1.0e6~2.5e7

- Stage 2: Final tuned circuit netlist

- File name must be “**final_design.cir**”
- Reference the “final_design_template_v2.cir” (see right), your final_design.cir should complete the TODO part. **Do not modify other parts!!**

```
1 two_stage_amp test
2
3 * Two stage OPAMP
4 .include /tmp/ngspice_model/45nm_bulk.txt
5
6 ***** TODO *****
7 .param wp1= lp1=90n mp1=
8 .param wn1= ln1=90n mn1=
9 .param wn3= ln3=90n mn3=
10 .param wp3= lp3=90n mp3=
11 .param wn4= ln4=90n mn4=
12 .param wn5= ln5=90n mn5=
13 .param cc=
14 *****
15
16 .param ibias=30u
17 .param cload=10p
18 .param vcm=0.6
19
20 mp1 net4 net4 VDD VDD pmos w=wp1 l=lp1 m=mp1
21 mp2 net5 net4 VDD VDD pmos w=wp1 l=lp1 m=mp1
22 mn1 net4 net2 net3 net3 nmos w=wn1 l=ln1 m=mn1
23 mn2 net5 net1 net3 net3 nmos w=wn1 l=ln1 m=mn1
24 mn3 net3 net7 VSS VSS nmos w=wn3 l=ln3 m=mn3
25 mn4 net7 net7 VSS VSS nmos w=wn4 l=ln4 m=mn4
26 mp3 net6 net5 VDD VDD pmos w=wp3 l=lp3 m=mp3
27 mn5 net6 net7 VSS VSS nmos w=wn5 l=ln5 m=mn5
28 cc net5 net6 cc
29 ibias VDD net7 ibias
30
31 vin in 0 dc=0 ac=1.0
32 ein1 net1 cm in 0 0.5
33 ein2 net2 cm in 0 -0.5
34 vcm cm 0 dc=vcm
35
36 vdd VDD 0 dc=1.2
37 vss 0 VSS dc=0
38 CL net6 0 cload
39 |
40 .end
41
```

5. Grading Policy

5.1. Stage 1: 70%

- Criterion: success rate of at least 85%.

5.2. Stage 2: 20%

- **2 public test cases, 10% for each.**
- Correctness: 5%
- Runtime performance: 5%

5.3. Report: 10%

6. Evaluation Detail

- **Correctness (5%):** The generated final circuit (final_design.cir) passes the checker. The checker uses NGSpice simulations to determine whether the final circuit meets the target specifications. **Please execute at work station 140.113.201.202 .**

```
$ conda activate autockt
$ python3 sizing_[student_ID].py --model model_[student_ID] --spec <spec
json>
$ /tmp/ngspice_model/ngspice_checker.py --netlist final_design.cir --
spec <spec json>
```

- **Runtime performance (5%):** Submissions are ranked by the time required to generate the final circuit; shorter time yields a higher score.
 - Suppose there are N valid submissions for a test case
 - Let $\text{rank}(i)$ denote the ranking of submission i ($1 = \text{best}$, $N = \text{worst}$).
 - The performance score S_i for submission i is
$$S_i = \frac{N - \text{rank}(i) + 1}{N}$$
 - Thus, the best submission ($\text{rank} = 1$) receives 5 points, and the worst submission ($\text{rank} = N$) still receives 5 N points.
- You can shorten the time to generate the circuit by studying how to speed up the RL agent, how to set an appropriate trajectory length, etc.

6.1. Violating file naming rules: **-10** points

6.2. For submitting the executable file: **-10** points

6.3. If **plagiarism** is detected, you will receive a score of **0**.

6.4. **Third-party solver** is prohibited.

6.5. The program must execute successfully on the official workstation.

6.6. **Late submission will get no grades!!**